Rachunek macierzowy - lab 2

[Piotr Sielchanowicz] [Jakub Karczewski] [Michał Zienkowicz]

31.03.2026

1 Treść zadania

Proszę zaimplementować dla macierzy o rozmiarze $N \times N$, gdzie $N = 25$ (miesiąc + dzień urodzenia):

1. Algorytm eliminacji Gaussa bez pivotingu generujący jedynki na przekątnej.
2. Algorytm eliminacji Gaussa z pivotingiem.
3. Algorytm LU faktoryzacji bez pivotingu.
4. Algorytm LU faktoryzacji z pivotingiem.

2 Wstęp teoretyczny

Jednym z fundamentalnych problemów algebry liniowej jest rozwiązywanie układów równań liniowych zapisanych w postaci macierzowej $Ax = b$, gdzie A jest macierzą współczynników, x wektorem niewiadomych, a b wektorem wyrazów wolnych. Metody rozwiązywania tych układów dzielą się na bezpośrednie (np. eliminacja Gaussa, rozkład LU) oraz iteracyjne. Metody bezpośrednie pozwalają na wyznaczenie dokładnego rozwiązania (z dokładnością do błędów zaokrągleń) w skończonej liczbie kroków.

3 Przedstawienie algorytmów

3.1 Algorytm Gaussa

3.1.1 Algorytm Gaussa - teoria

Algorytm opiera się na przekształceniu macierzy A do macierzy schodkowej U przy użyciu operacji elementarnych na wierszach. W wersji algorytmu z polecenia, każdy wiersz jest dzielony przez wartość elementu osiowego przed przystąpieniem do zerowania pozostałych (poniższych) elementów w kolumnie.

Algorytm Gaussa może zawieść lub zwracać niedokładne wyniki, gdy na przekątnej pojawi się zero lub liczba o bardzo niskiej wartości. Rozwiązaniem tego problemu jest pivoting – wybór elementu wiodącego. Polega on na znalezieniu w bieżącej kolumnie elementu o największej wartości bezwzględnej i zamianie wierszy tak, aby element ten znalazł się na pozycji osiowej. Operacja ta zwiększa precyzję obliczeń.

3.1.2 Algorytm Gaussa - implementacja (Python)

```python
def gauss_elimination(M, pivoting = True, normalize = True):
    n = len(M)

    for j in range(n):
        if pivoting:
            maxi, pivot_idx = 0.0, 0
            for i in range(j, n):
                if abs(M[i][j]) > maxi :
                    maxi, pivot_idx = abs(M[i][j]), i
            if pivot_idx != j: # we need to swap
                swap_rows(j, pivot_idx, M)
        if normalize:
            divide_row(j, M[j][j], M) # we want our row to have 1.0 as the element on the diagonal

        for k in range(j + 1, n):
            assert not np.isclose(M[j][j], 0)
            add_scaled_row_to_another(j, k, -M[k][j]/M[j][j], M)
```

3.1.3 Funkcje pomocnicze:

- `divide_row` - dzieli wszystkie elementy wskazanego wiersza przez zadaną wartość `k`.

```python
def divide_row(row, k, M):
    n = len(M)
    for j in range(n):
        M[row][j] /= k
```

- `swap_rows` - zmienia miejscami dwa podane wiersze macierzy.

```python
def swap_rows(row1, row2, M):
    n = len(M)
    for j in range(n):
        M[row1][j], M[row2][j] = M[row2][j], M[row1][j]
```

- `add_scaled_row_to_another` - dodaje do docelowego wiersza (`row2`) wartości z innego wiersza (`row1`) przemnożone przez współczynnik `k`.

```python
def add_scaled_row_to_another(row1, row2, k, M):
    n = len(M)
    for j in range(n):
        M[row2][j] += M[row1][j] * k
```

3.2 Faktoryzacja LU

Faktoryzacja LU polega na przedstawieniu macierzy kwadratowej A jako iloczynu dwóch macierzy trójkątnych – dolnej (L) i górnej (U). Można to zapisać jako $A = LU$ - wtedy rozwiązywany układ równań przyjmuje postać $LUx = b$. Macierz U jest standardowym wynikiem eliminacji Gaussa, natomiast macierz L budowana jest z współczynników używanych do zerowania elementów pod przekątną. Zaletą metody LU jest oszczędność czasu w sytuacji, gdy układ równań musi być rozwiązywany wielokrotnie dla różnych wektorów wyrazów wolnych – nie ma potrzeby powtarzać procesu eliminacji od zera.

Tak samo jak w metodzie Gaussa niskie wartości liczb na przekątnych mogą powodować niestabilność numeryczną. Tak jak w pivotingu w standardowym algorytmie Gaussa, wersja faktoryzacji LU z pivotingiem rozwiązuje ten problem poprzez przeszukiwanie bieżącej kolumny i celową zamianę wierszy tak, aby kluczowym dzielnikiem zawsze była liczba o największej możliwej wartości bezwzględnej. Historia zamian jest notowana w macierzy permutacji (P). Modyfikuje to wzór do postaci $PA = LU$.

3.2.1 Faktoryzacja LU - implementacja (Python)

```python
def lu_factorization(A, pivoting = True):
    n = len(A)
    P = [i for i in range(n)]
    A = np.array(A, dtype=np.float32)
    L, U = np.array([[0.0 for _ in range(n)] for _ in range(n)]), np.copy(A)
    for i in range(n):
        L[i, i] = 1.0
    for i in range(n):

        if pivoting or np.isclose(U[i, i], 0.0):
            if pivoting:
                pivot_idx = i + np.argmax(np.abs(U[i:, i]))
            else:
                pivot_idx = i + np.where(~np.isclose(np.abs(U[i:, i]), 0.0))[0][0]

            if pivot_idx != i:
                U[[i, pivot_idx]] = U[[pivot_idx, i]]
                L[[i, pivot_idx], :i] = L[[pivot_idx, i], :i]
                P[i], P[pivot_idx] = P[pivot_idx], P[i]

            assert not np.isclose(U[i, i], 0.0)

        for j in range(i+1, n):
            factor = U[j][i]/U[i, i]
            U[j, :] -= factor * U[i, :]
            L[j, i] = factor

    P = np.array(P)
    P_res = np.zeros((P.size, P.max() + 1))
    P_res[np.arange(P.size), P] = 1
    return P_res, L , U
```

4 Porównanie czasu obliczeń dla zestawów równań liniowych różniących się wektorem wyrazów wolnych

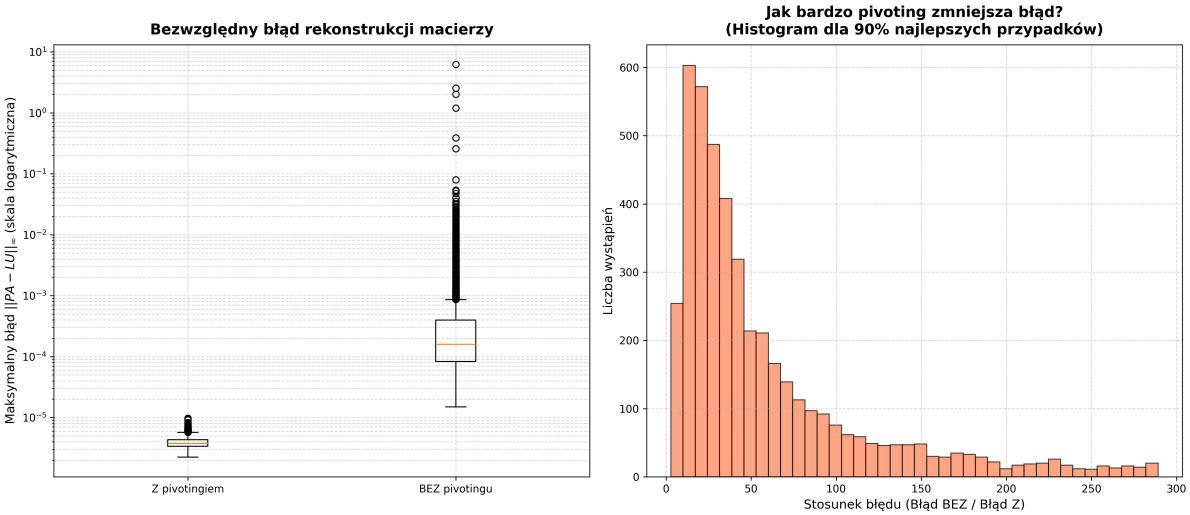
Rysunek 1: Porównanie czasów działania algorytmów w zależności od ilości wektorów wyrazów wolnych.

Pomimo, że obie metody rozwiązywania układów równań liniowych mają jednakowy rząd złożoności obliczeniowej $O(n^3)$, za pomocą faktoryzacji LU możliwe jest rozwiązanie wielu układów równań znacznie szybciej niż metodą eliminacji Gaussa, gdy układy te różnią się jedynie wektorami wyrazów wolnych. Wynika to z faktu, że najbardziej kosztowny etap algorytmu LU – sam rozkład macierzy rzędu $O(n^3)$ – wykonywany jest tylko jeden raz. Wyznaczenie wyniku dla każdego kolejnego wektora b wymaga już jedynie operacji podstawiania w przód i wstecz, których złożoność wynosi zaledwie $O(n^2)$. W przeciwieństwie do tego, klasyczna eliminacja Gaussa wymusza powtarzanie całego procesu $O(n^3)$ od nowa dla każdego badanego wektora.

5 Porównanie dokładności obliczeń dla pivotingu i braku pivotingu - dla faktoryzacji LU

Eksperyment wykonano na zbiorze 5000 losowych, gęstych macierzy o wymiarach 40×40 z wartościami z zakresu $0 - 10$. Każda macierz została poddana faktoryzacji za pomocą algorytmu bez pivotingu oraz algorytmu z pivotingiem.

Za miarę dokładności przyjęto maksymalny błąd bezwzględny pomiędzy zrekonstruowaną macierzą LU a oryginalną macierzą układu po permutacji (norma $||PA - LU||$).



Rysunek 2: Porównanie dokładności dla algorytmu LU z pivotingiem i bez pivotingu.

Z wykresów widać bardzo silną redukcję maksymalnego błędu bezwzględnego na skutek zaimplementowania pivotingu. W przypadku algorytmu bez pivotingu, maksymalny błąd w każdym przypadku przekraczał błędy zwracane przez algorytm z pivotingiem. Często stosunek tych błędów był rzędu dziesiątek, czasem nawet setek.